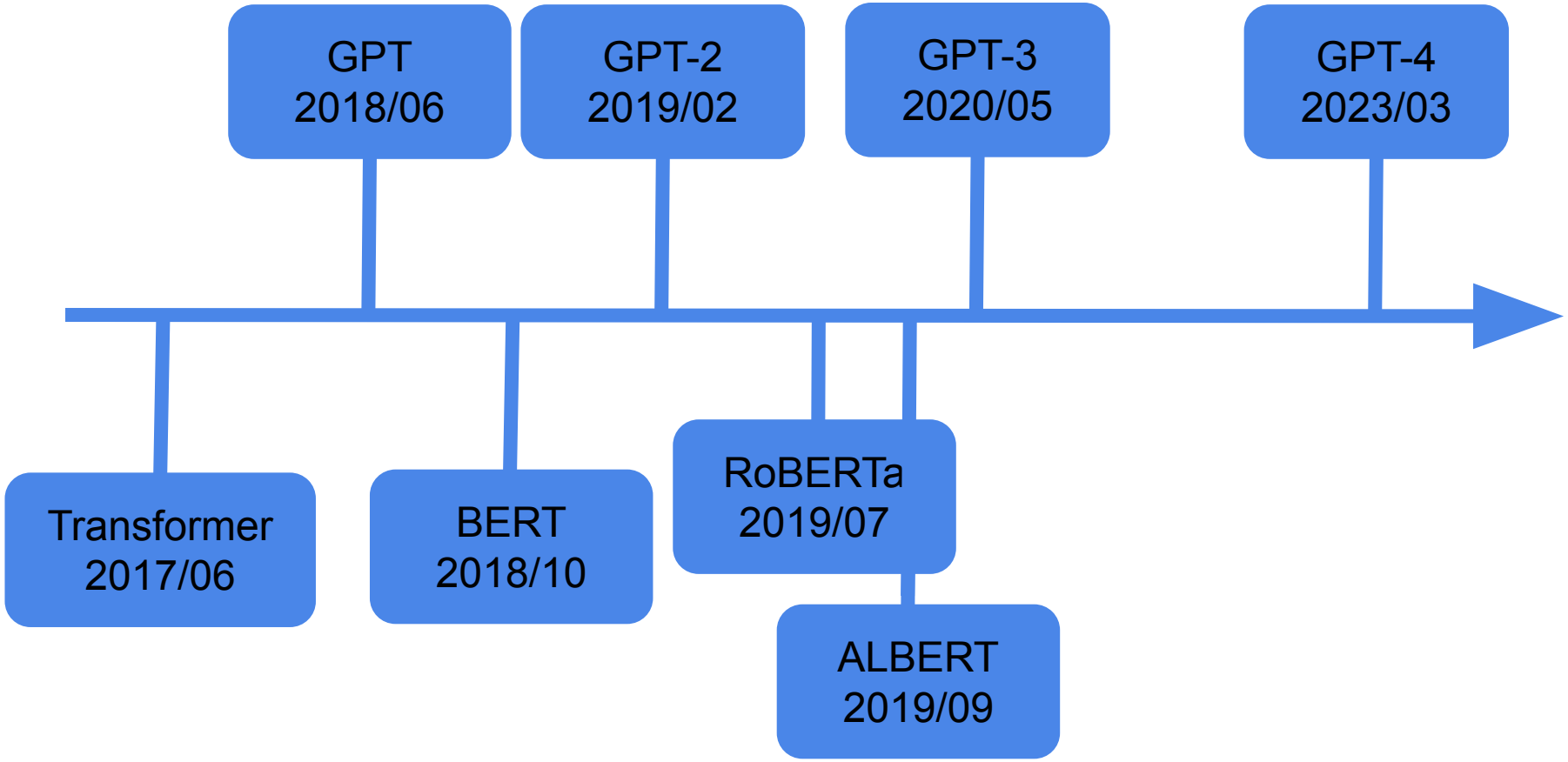
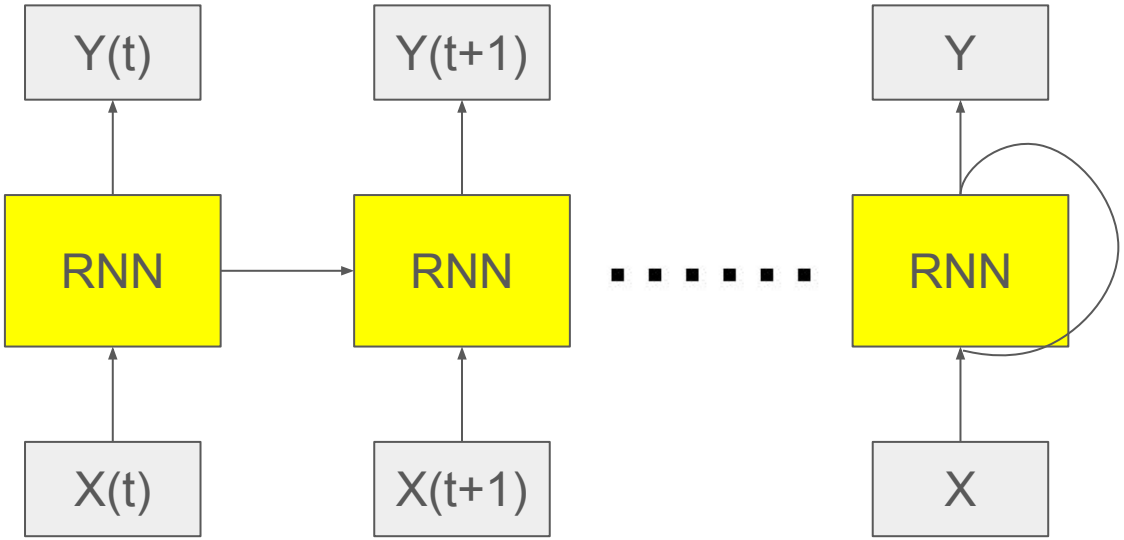
Transformer


GPT
2018/06
GPT-2
2019/02
GPT-3
2020/05
GPT-4
2023/03
Transformer
2017/06
BERT
2018/10
RoBERTa
2019/07
ALBERT
2019/09

RNN



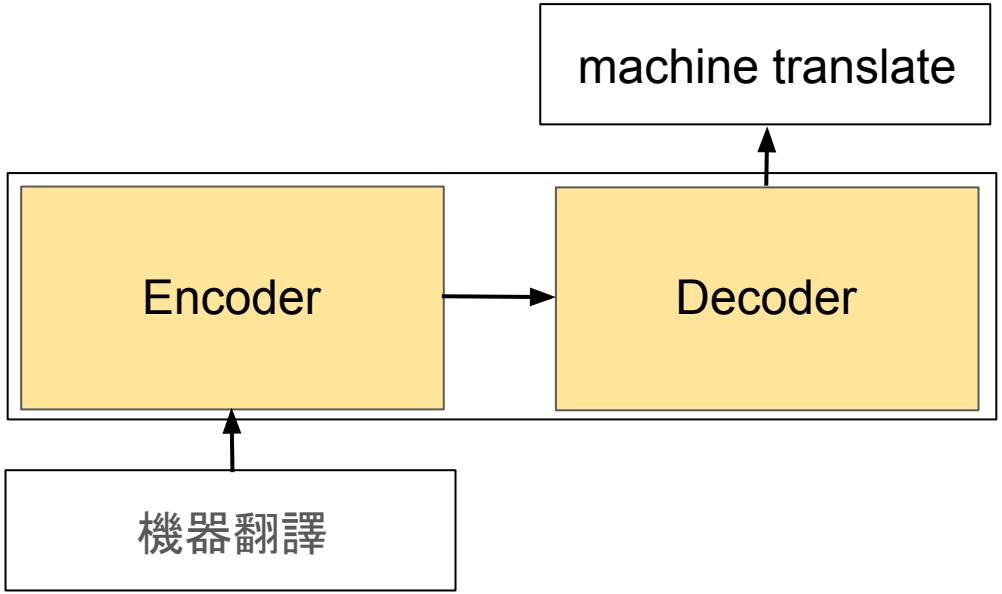
- 每個輸出都要依賴前一個輸出，因此在進行反向訓練時，都會乘上參數越乘越大會造成**梯度爆炸**，導致模型不穩定甚至無法訓練
- 反之，則造成**梯度消失**，導致權重幾乎沒改變

RNN

	RNN	Transformer
並行處理能力	RNN處理下一個詞必須知道前一個詞的結果，因此難以進行並行計算	Transformer使用**注意力機制**，可以同時處理整個序列的所有元素
長序列距離依賴問題	由於梯度消失和梯度爆炸問題，難以捕捉到序列中長距離詞之間的依賴關係。 儘管LSTM和GRU等RNN變體都設計來處理梯度消失問題，但實際效果仍有限	注意力機制使模型直接關注序列中任意兩個位置之間的關係，無論距離多遠，使得它能更好地捕捉長距離關係

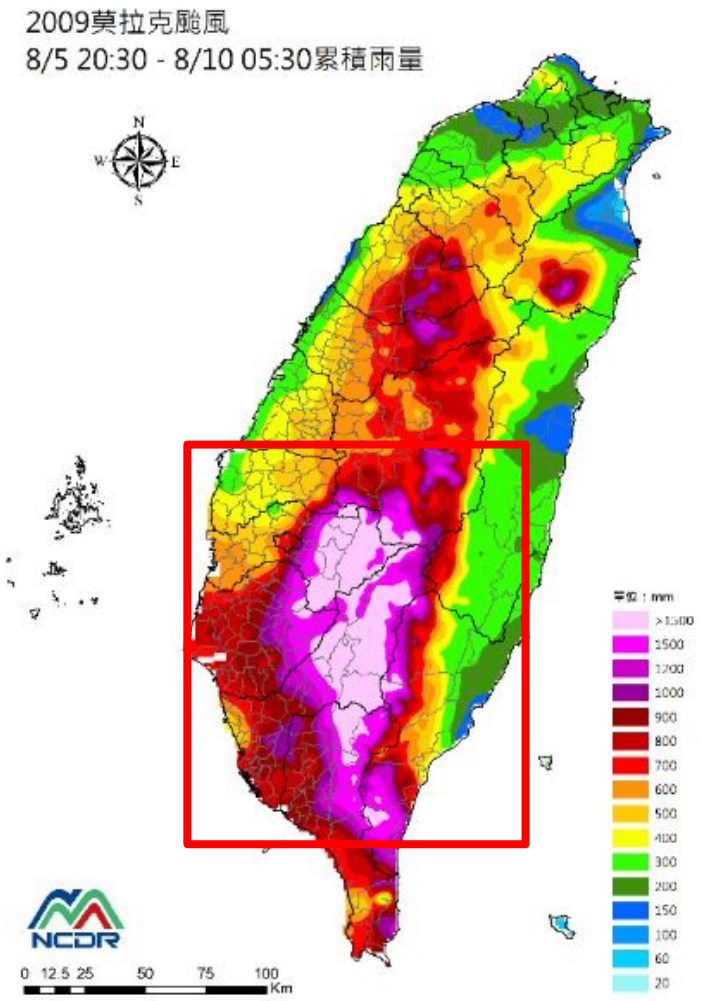
Transformer

- 於2017年推出，將原始的RNN架構改成 Attention mechanism 同時處理輸入序列的所有位置，並通過多頭注意力等機制捕捉不同層次的特徵
- Transformer 的 Attention mechanism 允許所有位置的計算都可以並行進行

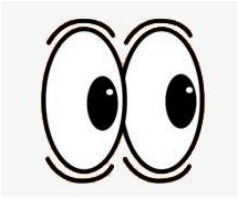


Attention mechanism

當第一眼看到這張圖，注意力集中在哪裡



Attention mechanism



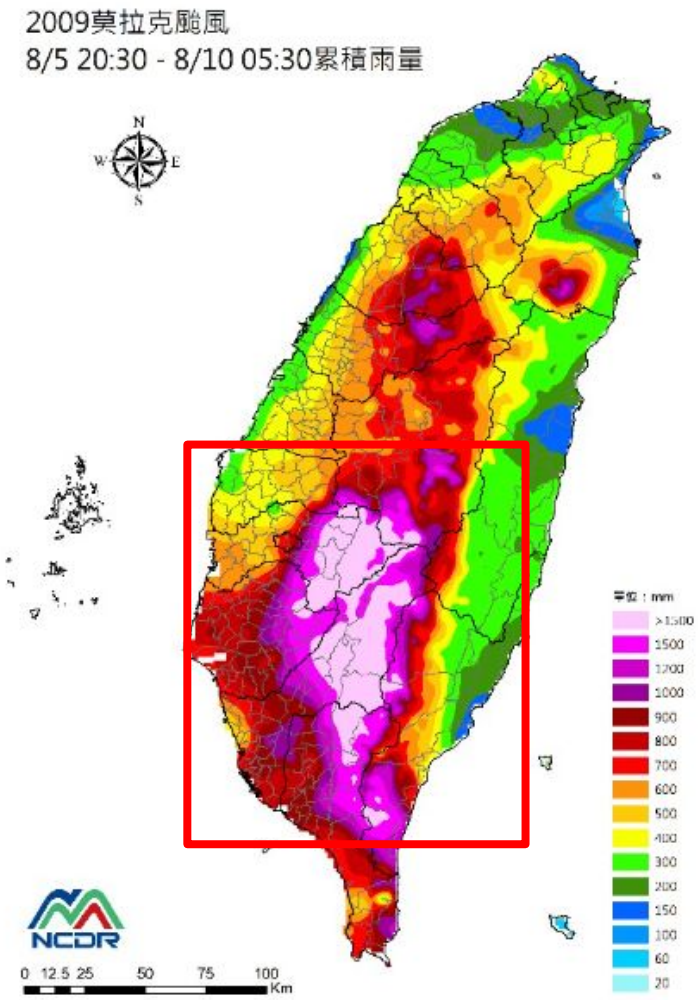
接收訊息

整張圖片

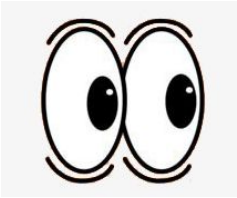
Sensory memory

Attention(放入重要的)

Working memory



Attention mechanism



接收訊息

整張圖片

Sensory memory

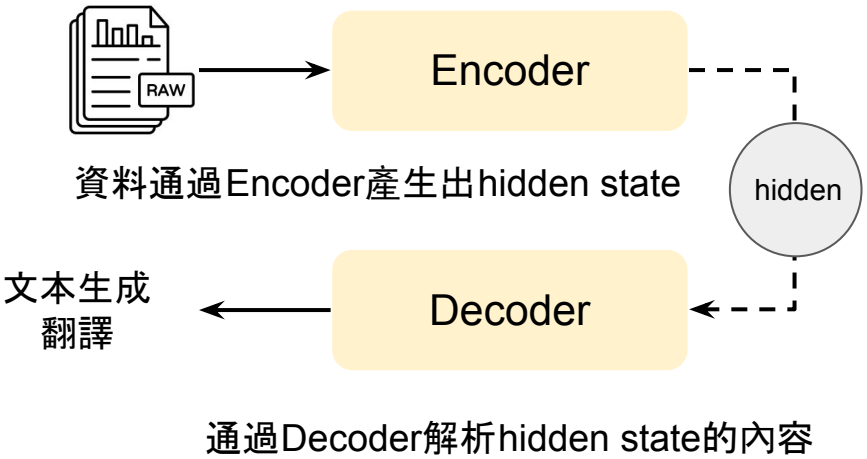
Attention(放入重要的)

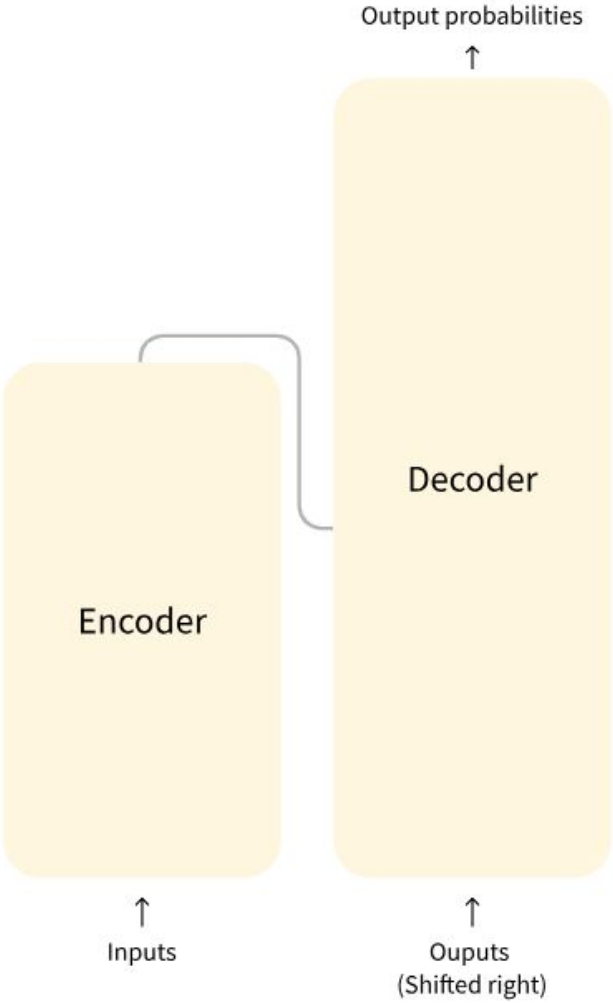
Working memory

如何依照輸入的重要性
給予相應的權重

Machine translation

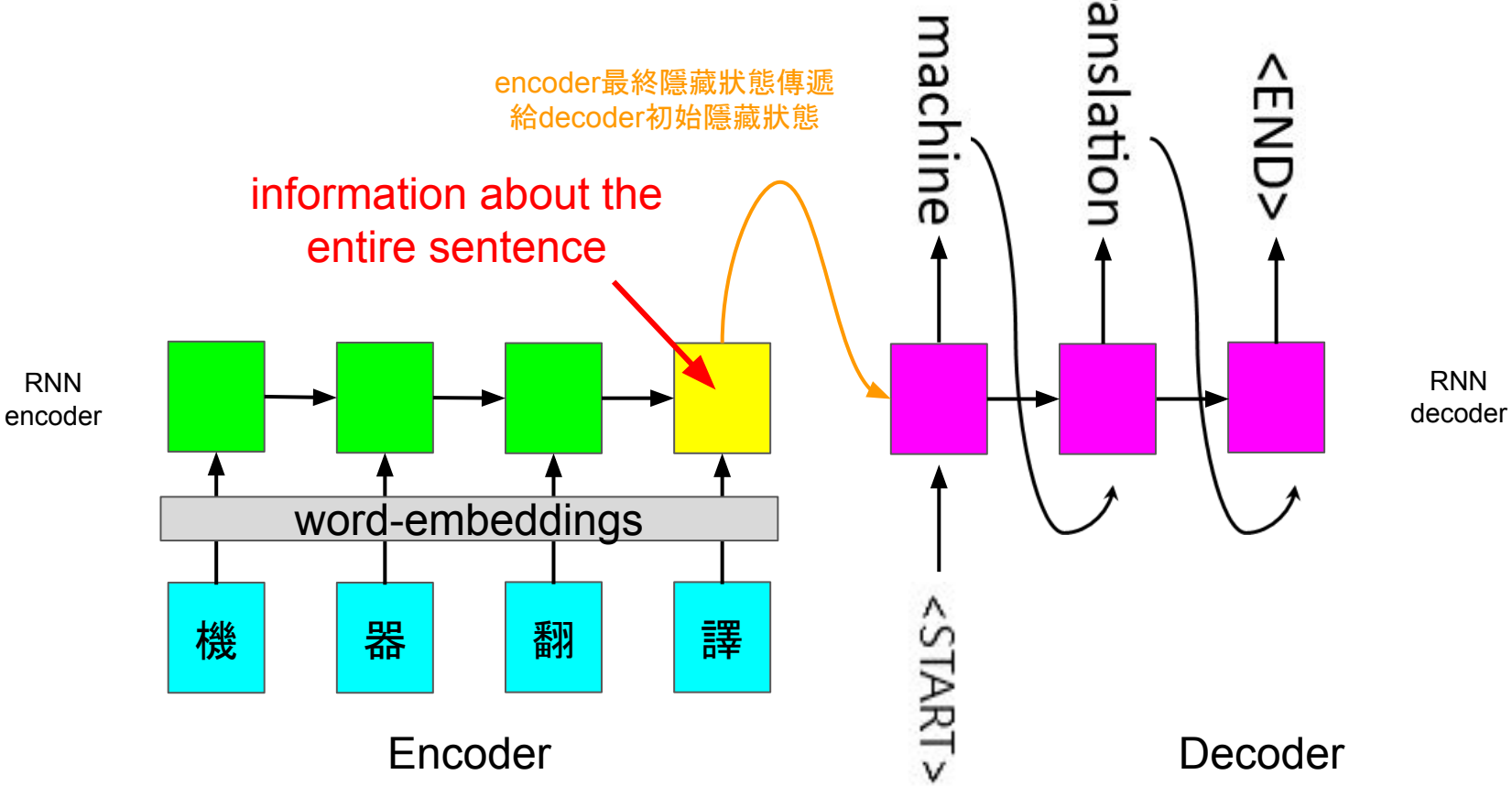
- Sequence-to-sequence（encoder - decoder model）





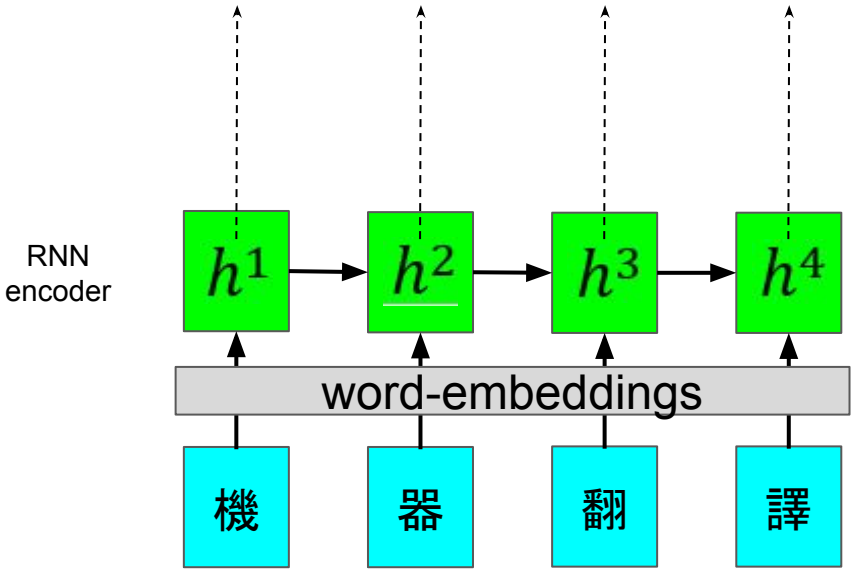
source: https://huggingface.co/learn/nlp-course/zh-TW/chapter1/4?fw=pt

Machine translation without attention



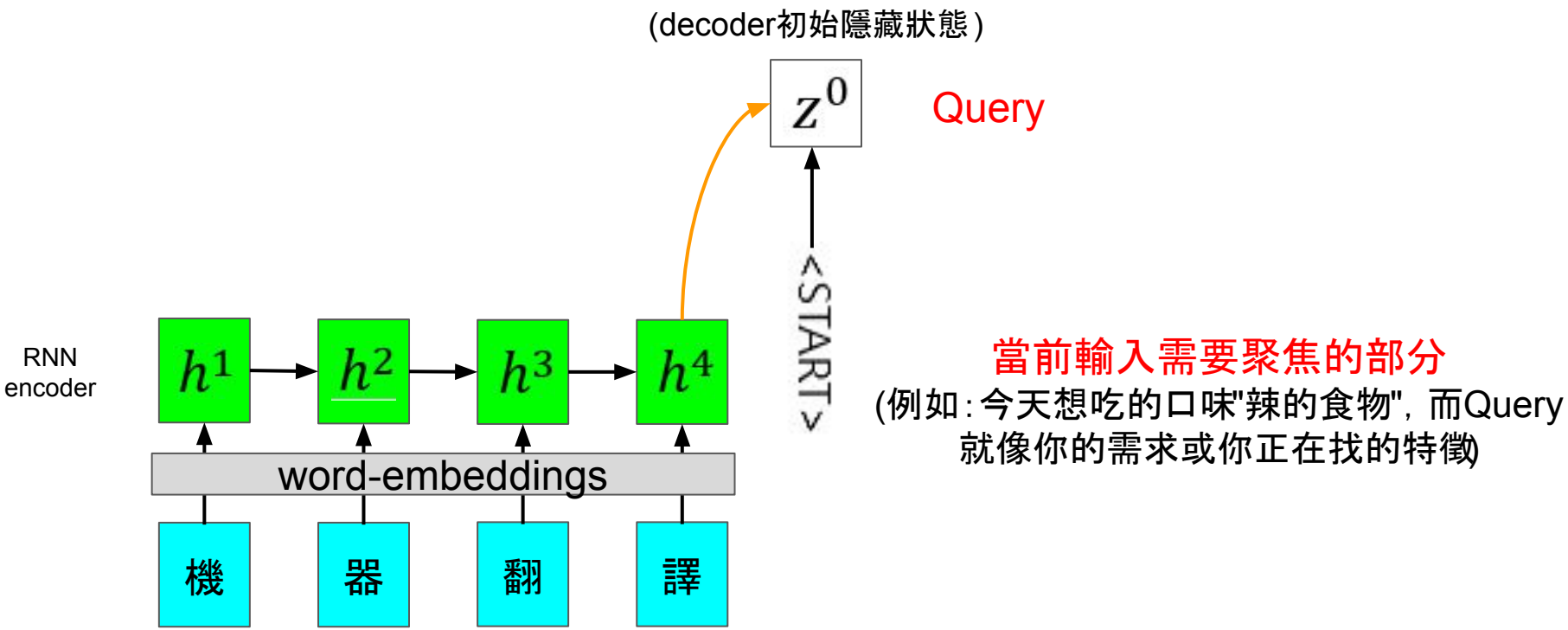
Machine translation with attention

每個隱藏狀態都會生成該詞的"key"跟”value”

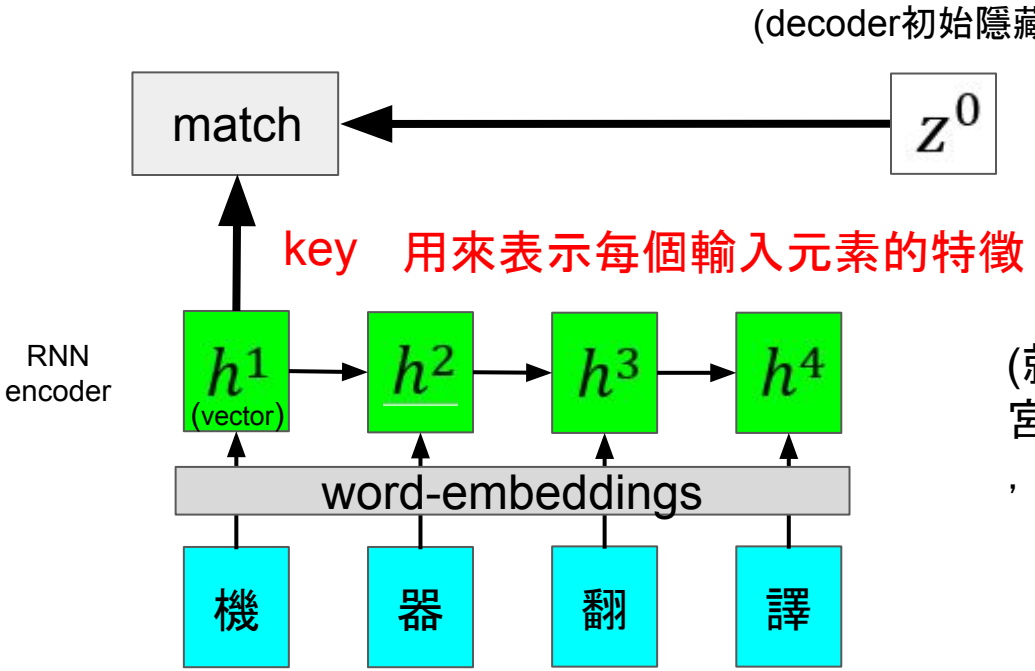


這些 Key 和 Value 就是後續 Attention 機制會用來做「匹配」與「加權平均」的依據。

Machine translation with attention

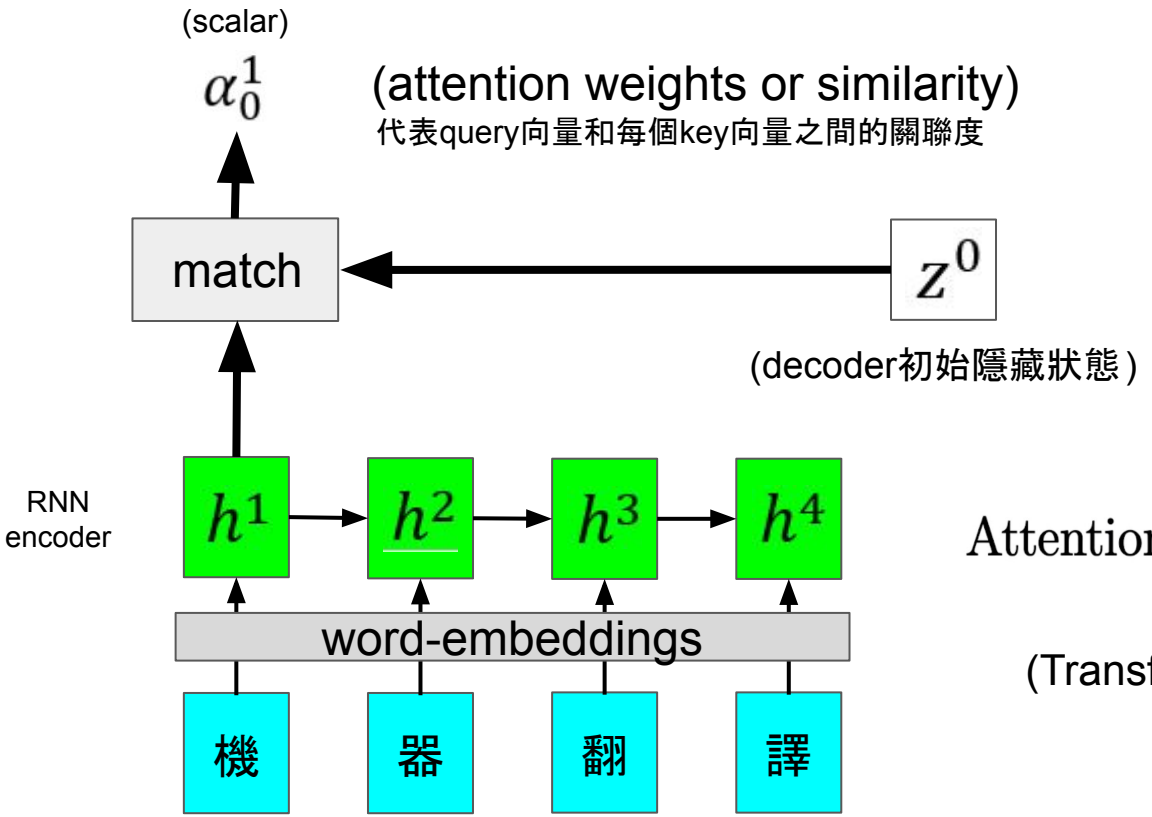


Machine translation with attention



(就像菜單上每道菜的敘述，例如：宮保雞丁-"辣"、糖醋排骨-"甜"，key代表每道菜的特徵或標籤)

Machine translation with attention



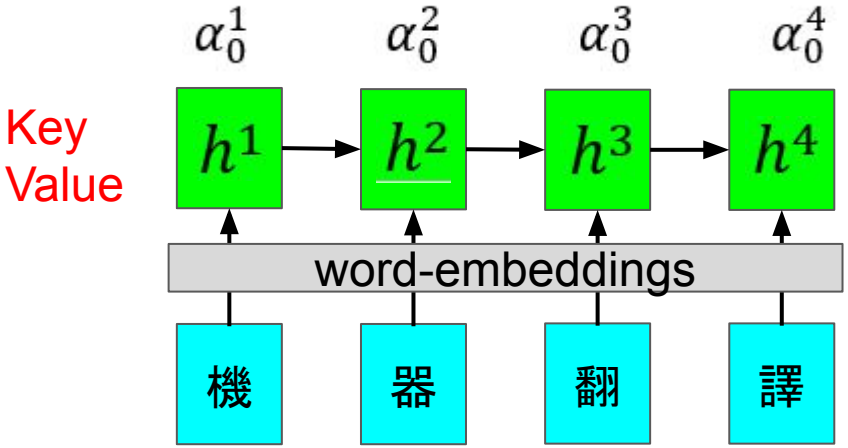
$$\text{Attention Weights} = \frac{QK^T}{\sqrt{d_k}}$$

(Transformer version)

Machine translation with attention

z^0

Query



Value向量代表輸入的實際內容
(菜單上每道的實際內容，當選擇了一道菜，這道菜(宮保雞丁)就是Value)

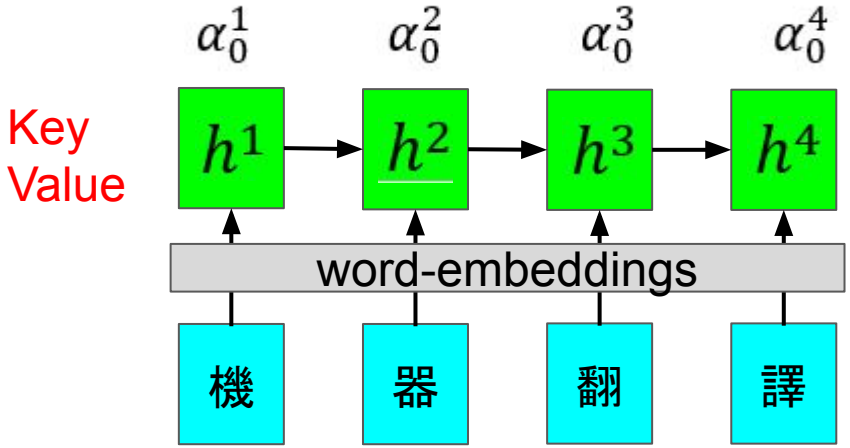
$$attention(Q, K, V) = \sum_{i=1}^{n} similarity(Q, K_i) * V_i$$

Machine translation with attention

Note: RNN的key和value為同樣的h，後續的transformer才改良成 h_k, h_v

z^0

Query



Value向量代表輸入的實際內容
(菜單上每道的實際內容，當選擇了一道菜，這道菜(宮保雞丁)就是Value)

$$attention(Q, K, V) = \sum_{i=1}^{n} similarity(Q, K_i) * V_i$$

Machine translation with attention

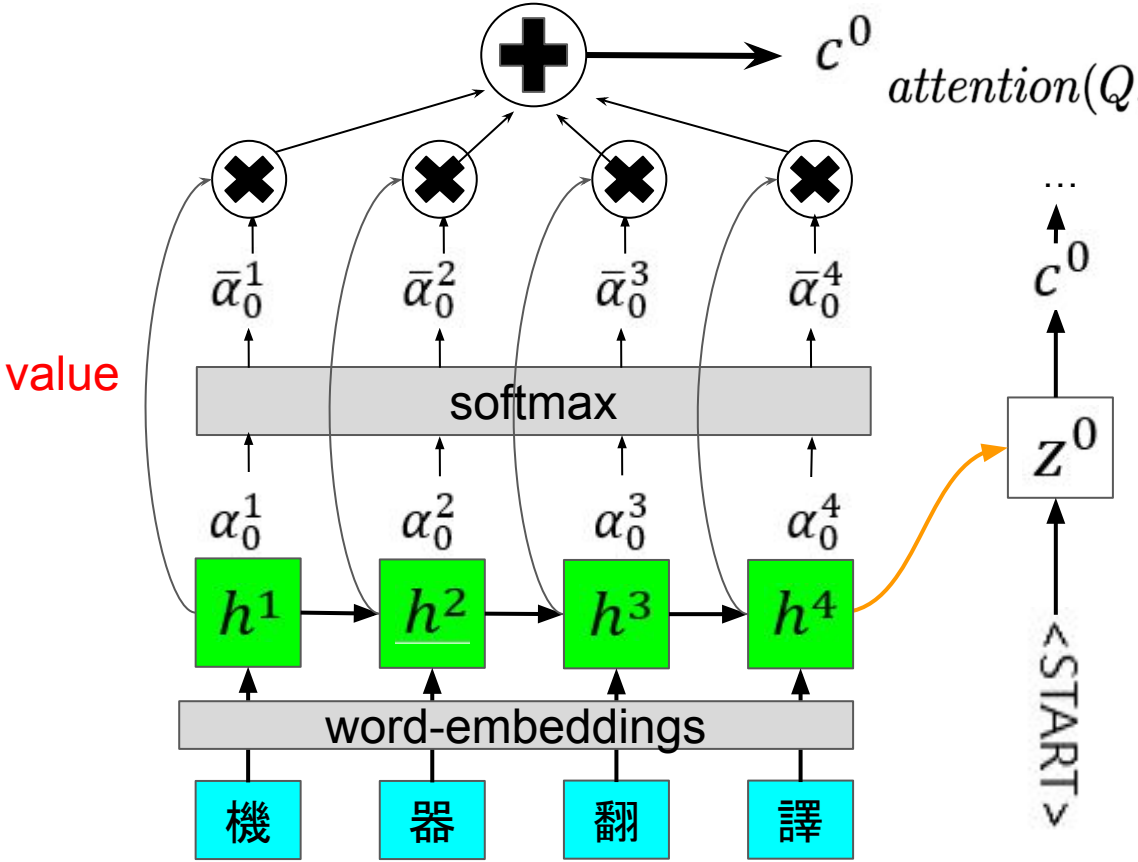
"黃金獵犬是一種非常聰明且友好的狗"

當Query是"狗"這個詞，可能會關注到 "黃金獵犬"和"友好"兩個詞：

- **黃金獵犬**的key可能對應〝狗的一種〞的特徵，而Value則是具體的〝黃金獵犬〞這個含義
- **友好**的key可能對有〝狗的特性〞的特徵，而Value則是具體〝友好〞這個詞的含義

計算得到"黃金獵犬"和"友好"的權重後，注意力機制會用這些權重對Value進行加權平均，得到一個詞的上下文向量 ，c^0向量捕捉"狗"這個詞在上下文中的含意

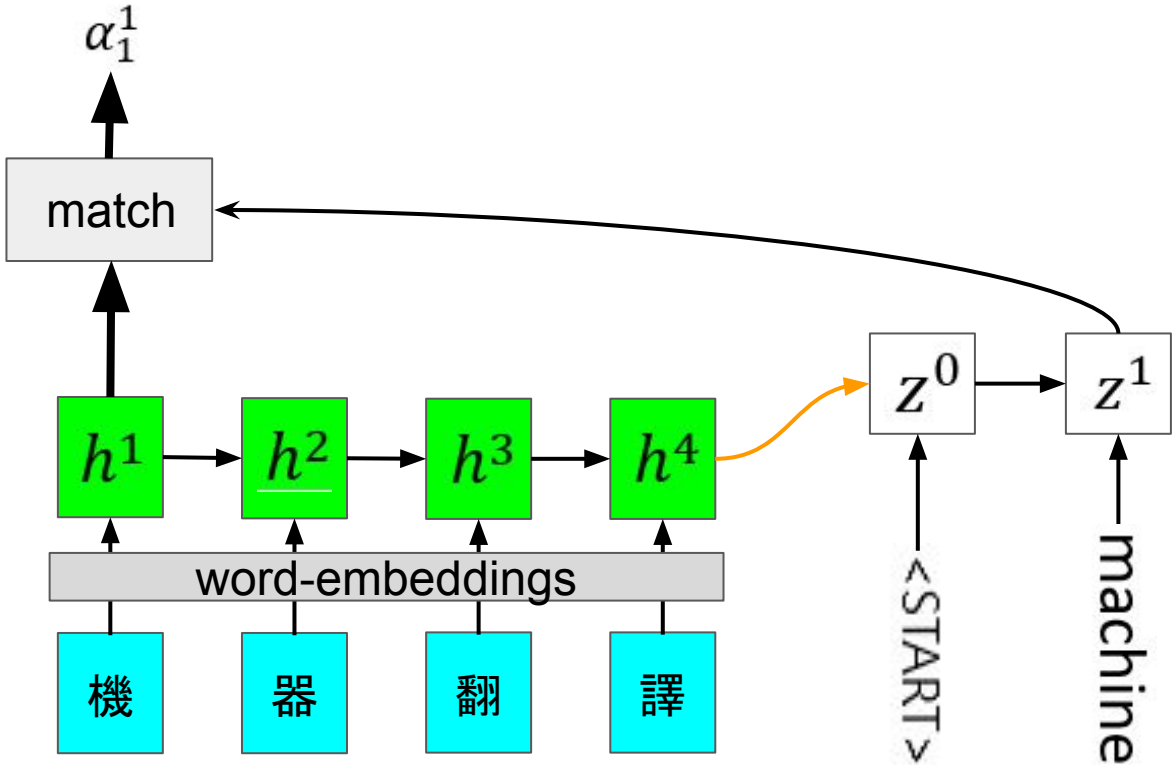
Machine translation with attention



$$attention(Q, K, V) = \sum_{i=1}^{n} similarity(Q, K_i) * V_i$$

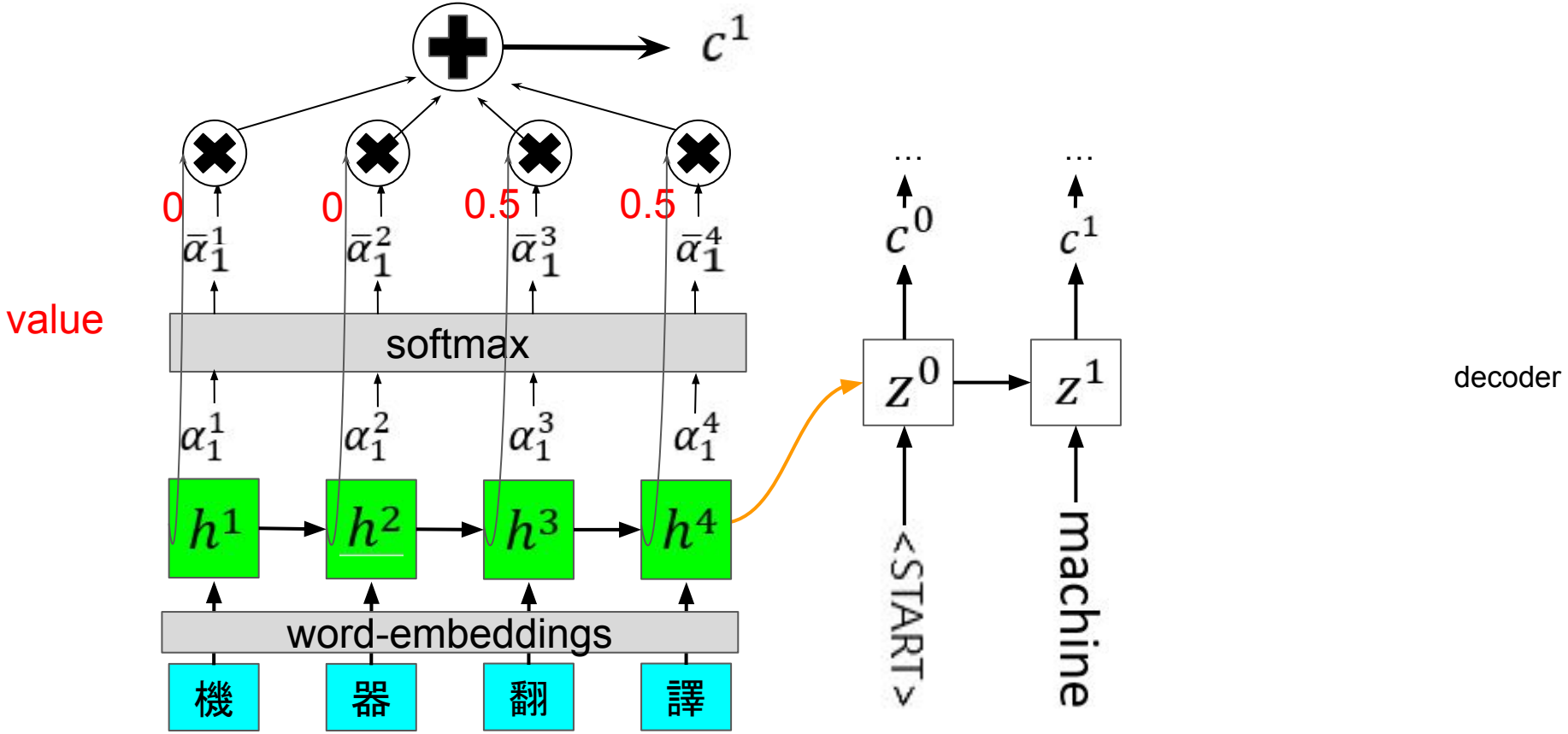
decoder

Machine translation with attention

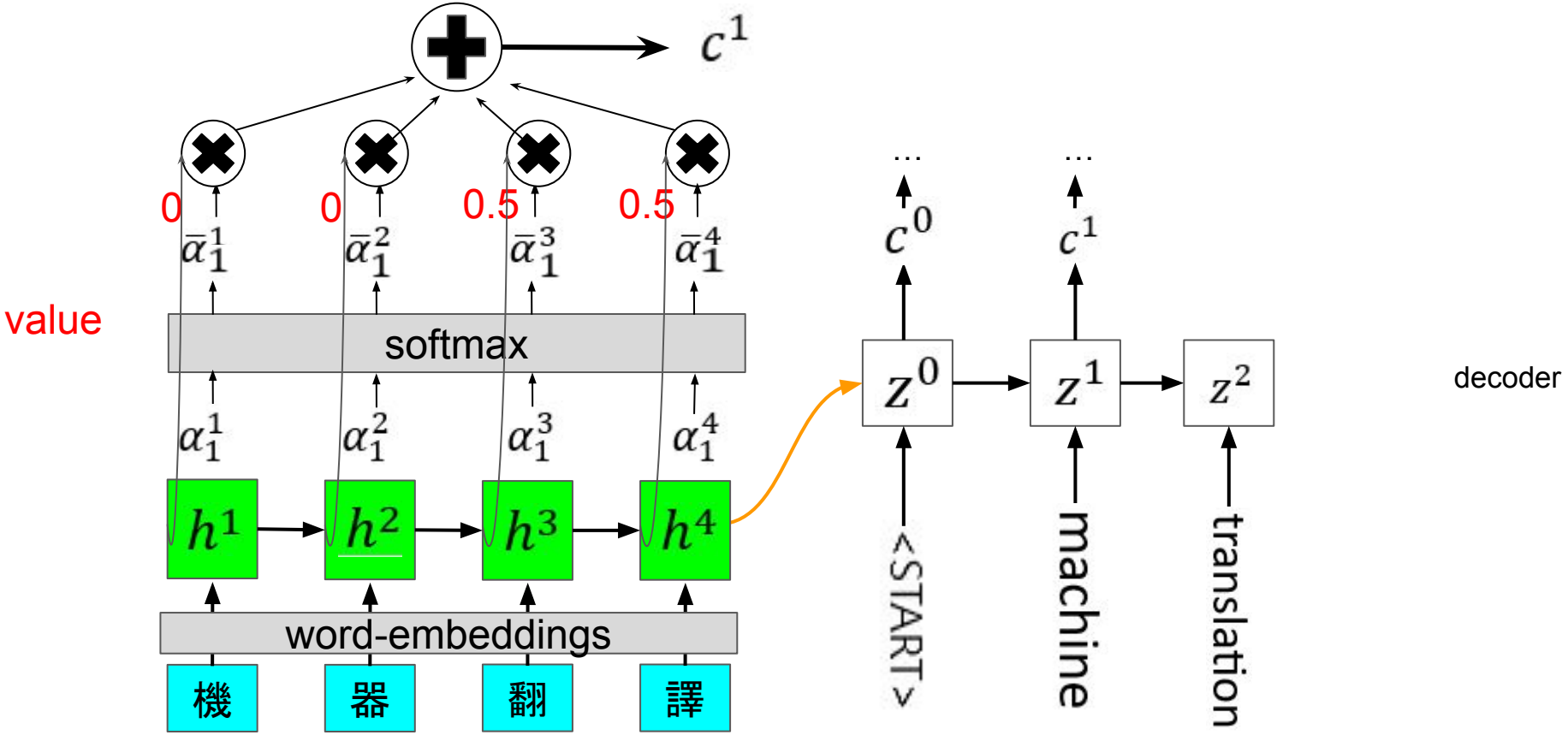


decoder

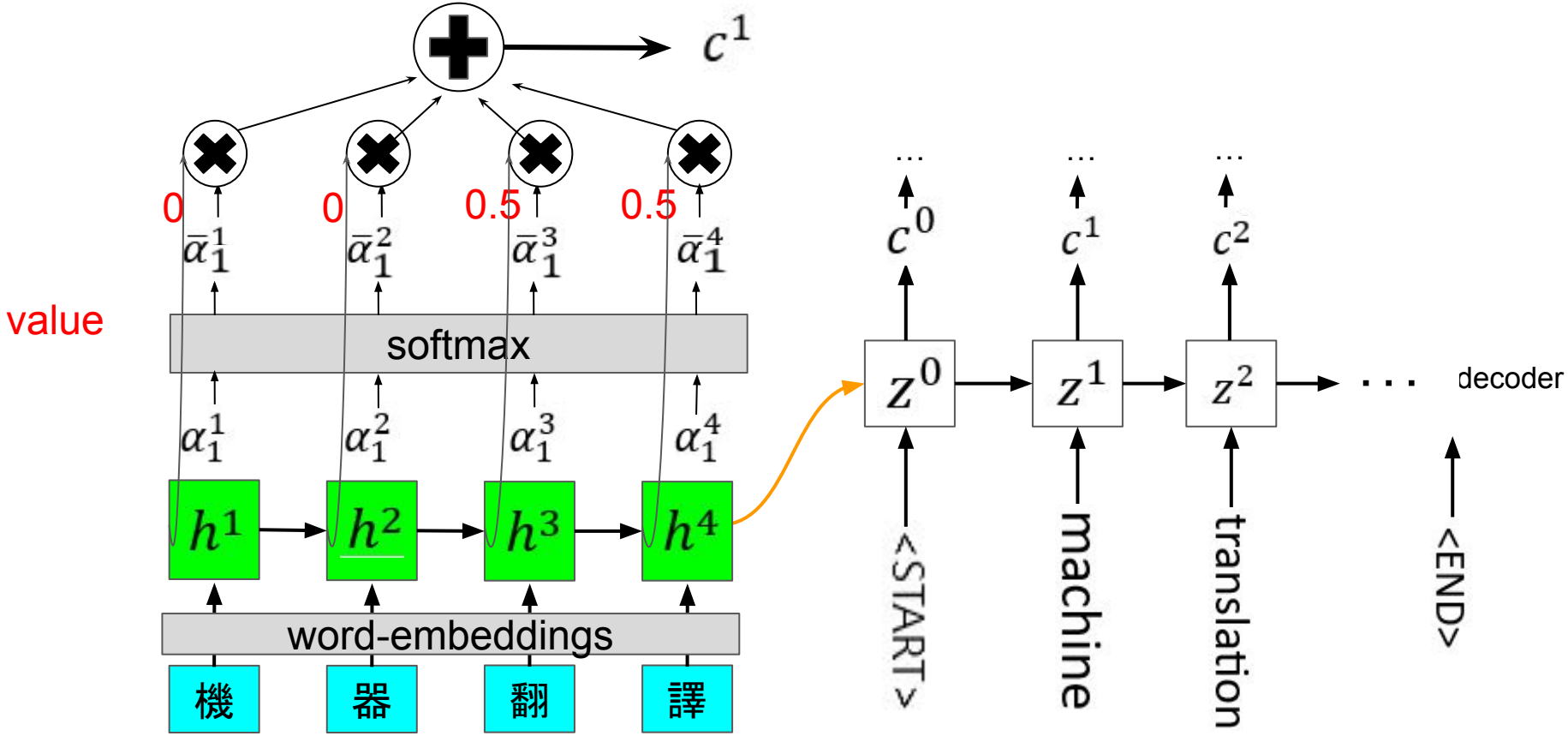
Machine translation with attention



Machine translation with attention



Machine translation with attention



Self-Attention

“Self-Attention 是一種讓模型在處理序列時，根據句子中其他詞的重要性來重新表示每個詞的方法。”

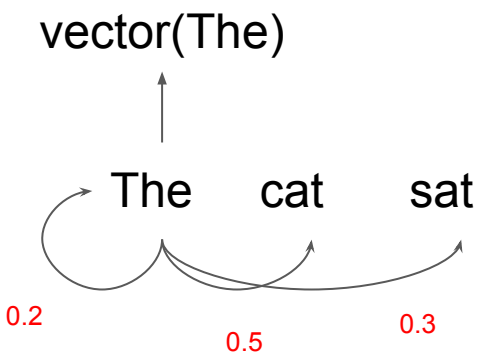
- 每個詞都會：
  - 對自己和其他詞打分數（attention weights）
  - 加權平均其他詞的資訊，來重建自己的表示
- 範例：輸入句子：The cat sat

Token	建立表示時會看哪些詞？
The	The, cat, sat
cat	The, cat, sat
sat	The, cat, sat

Self-Attention 計算流程

- 每個 token 都會變成三個向量:
  - Query (Q): 我要注意什麼？
  - Key (K): 我是什麼樣的資訊？
  - Value (V): 我提供什麼資訊？
- 對每個 token, Attention 的輸出是:
  - $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \times V$ (same as previous)

Self-Attention



每個 token 都根據全句的語境來重新理解自己

Multi-Head Attention

- 把「Self-Attention」這個動作做好幾次（多個 head），每次用不同的視角，然後再把結果組合起來。
  - 用不同角度去看一件事情，然後把觀察結果綜合起來做出更聰明的判斷。
- 單一的 self-attention 只能學到一種語意關係（例如主詞和動詞之間的連結），但語言中常常有很多不同層次的關係（例如語氣、句法結構、同義詞、地點關聯……）。
  - **Multi-head attention 讓模型可以「平行」學會多種關聯。**

How

- 把輸入的向量（例如每個詞的表示）
  → 切成多個子空間（例如從 512 維切成 8 組，每組 64 維）
- 每個 head：
  - 各自做一次 self-attention（使用自己的 Q, K, V）
  - 得到一組輸出表示
- 把所有 head 的輸出合併，再經過一層線性轉換來融合特徵（並確保輸出與原始維度一致）

Example

輸入句子:「The cat sat on the mat」

- head 1:學到「主詞與動詞」的關係
- head 2:學到「介系詞短語」的結構
- head 3:學到「the」通常修飾什麼詞
- …(總共有 8 或 12 個頭)

  → 最後把這些關係理解整合在一起，提升整體表示力

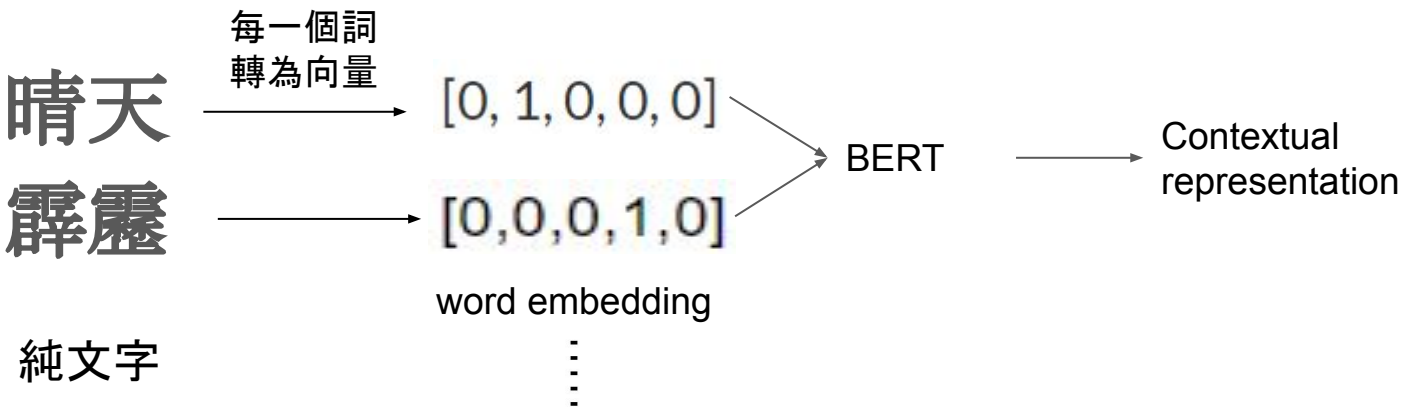
Comparison

類型	Query 來源	Key / Value 來源	是否看自己	常見位置	說明
Attention (cross)	一個序列	另一個序列	X	Decoder	輸出序列（如翻譯結果）根據輸入序列（原文）的資訊來決定該關注什麼
Self-Attention	序列自身	序列自身	V	Encoder / Decoder	每個詞都看整個句子，包括自己，用來捕捉上下文關係
Multi-Head Attention	多組分開的 Q/K/V（切分後的向量）	同上（依附於 self 或 cross）	V/X	Encoder / Decoder	把 Attention 機制做多次，從多種角度學語意，提升模型表示力（可用於 self 或 cross）

BERT

什麼是 BERT？

- BERT（Bidirectional Encoder Representations from Transformers）是 Google 所開發的預訓練語言模型。
- 它採用 Transformer Encoder 架構，搭配 Multi-Head Self-Attention，讓每個詞能根據整句話中的其他詞來重新建構自身表示，進而同時捕捉前後文的語意關係。
- BERT 使用 雙向語境理解 搭配 非監督式預訓練，可廣泛應用於各種自然語言任務，如文字標記、分類、問答等。



BERT

由於BERT進行編碼標記時考慮了雙向性，較不容易出現同一個字在不同語境(context)下產生歧義的問題

BERT

由於BERT進行編碼標記時考慮了雙向性，較不容易出現同一個字在不同語境(context)下產生歧義的問題

- 植物需要吸收「水分」
- 財務報表裡有「水分」

「水分」乍看之下是相同的詞，只是在不同的情境中具有不同的意義

BERT

由於BERT進行編碼標記時考慮了雙向性，較不容易出現同一個字在不同語境(context)下產生歧義的問題

- 植物需要吸收「水分」 ⟶ 植物為了生長所需的水分
- 財務報表裡有「水分」 ⟶ 資訊上的不實與虛假

「水分」乍看之下是相同的詞，只是在不同的情境中具有不同的意義

BERT

由於BERT進行編碼標記時考慮了雙向性，較不容易出現同一個字在不同語境(context)下產生歧義的問題

- 植物需要吸收「水分」 ⟶ 植物為了生長所需的水分
- 財務報表裡有「水分」 ⟶ 資訊上的不實與虛假

「水分」乍看之下是相同的詞，只是在不同的情境中具有不同的意義

word2vec：

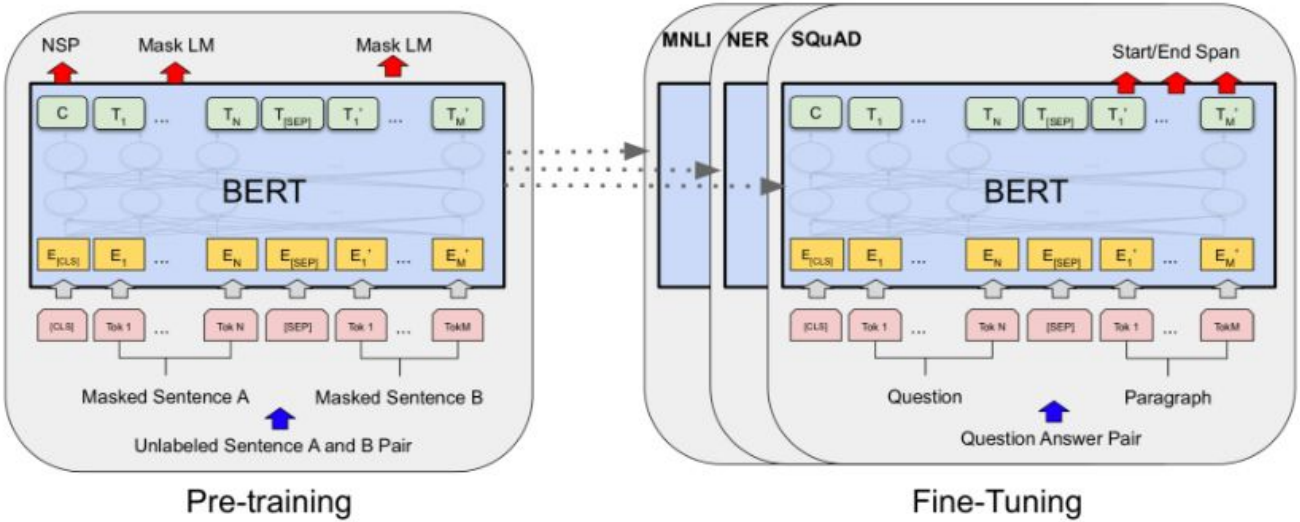
不考慮上下文進行編碼的word2vec模型因word vector都一樣，因此無法分辨而產生歧異

BERT：

BERT因會考慮上下文進行編碼，故植物「水分」與財務報表「水分」的word vector不一樣，則不容易產生歧異

BERT 如何做訓練？

- 在大型未標記的文字語料庫(非監督或半監督式學習)訓練語言模型 (pre-training)
- 針對特定的NLP任務微調此大型模型，利用此模型獲得的大型知識庫(fine-tuning)



source:https://arxiv.org/pdf/1810.04805.pdf

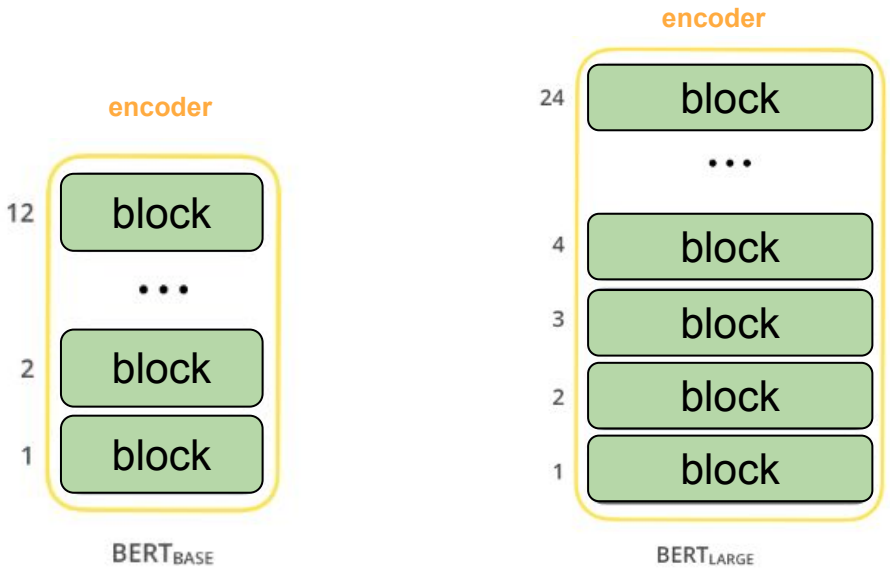
BERT 如何做訓練？

預先訓練模型時，必須有充足的語料才能確保訓練的全面性，內容的豐富度是BERT模型判斷的關鍵因素之一。因此，BERT模型通常在非常大型的語料庫上進行預先訓練，例如 Google 使用 BookCorpus 的 800M 詞和英文維基百科 2500M 詞

預訓練後面會提到！

BERT 主要模型

- BERT Base：12層編碼器區塊(Encoder blocks)，12個attention heads，110M個參數
- BERT Large：24層編碼器區塊(Encoder blocks)，16個attention heads，340M個參數

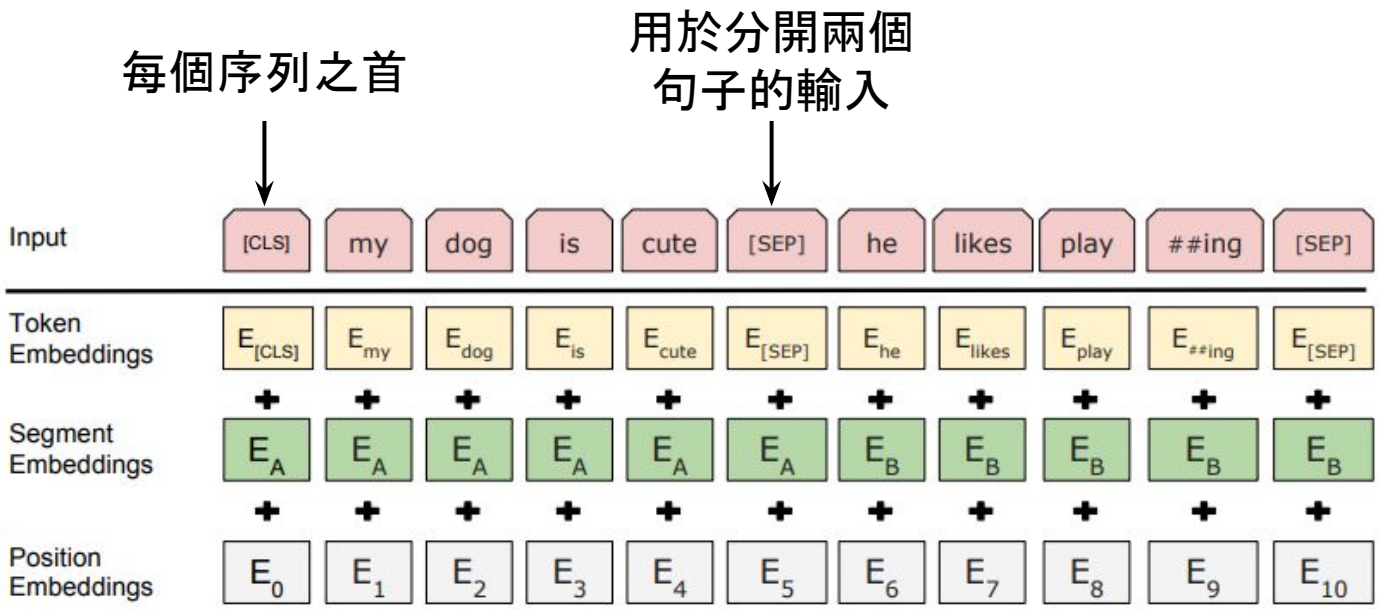


BERT 模型

Google開源了多個版本多個語言的預訓練模型於Github上

- BERT-Large, Uncased（Whole Word Masking）
  語言種類：英文
  網路結構：24-layer，1024-hidden，16-heads
  參數規模：340M
- BERT-Base, Uncased
  語言種類：英文
  網路結構：12-layer，768-hidden，12-heads
  參數規模：110M
- BERT-Large, Uncased
  語言種類：英文
  網路結構：24-layer，1024-hidden，16-heads
  參數規模：340M
- BERT-Large, Cased（Whole Word Masking）
  語言種類：英文
  網路結構：24-layer，1024-hidden，16-heads
  參數規模：340M
- BERT-Base, Cased
  語言種類：英文
  網路結構：12-layer，768-hidden，12-heads
  參數規模：110M
- BERT-Large, Cased
  語言種類：英文
  網路結構：24-layer，1024-hidden，16-heads
  參數規模：340M
- BERT-Base, Multilingual Cased
  語言種類：104種語言
  網路結構：12-layer，768-hidden，12-heads
  參數規模：110M
- BERT-Base, Multilingual Uncased
  語言種類：102種語言
  網路結構：12-layer，768-hidden，12-heads
  參數規模：110M
- BERT-Base, Chinese
  語言種類：中文
  網路結構：12-layer，768-hidden，12-heads
  參數規模：110M

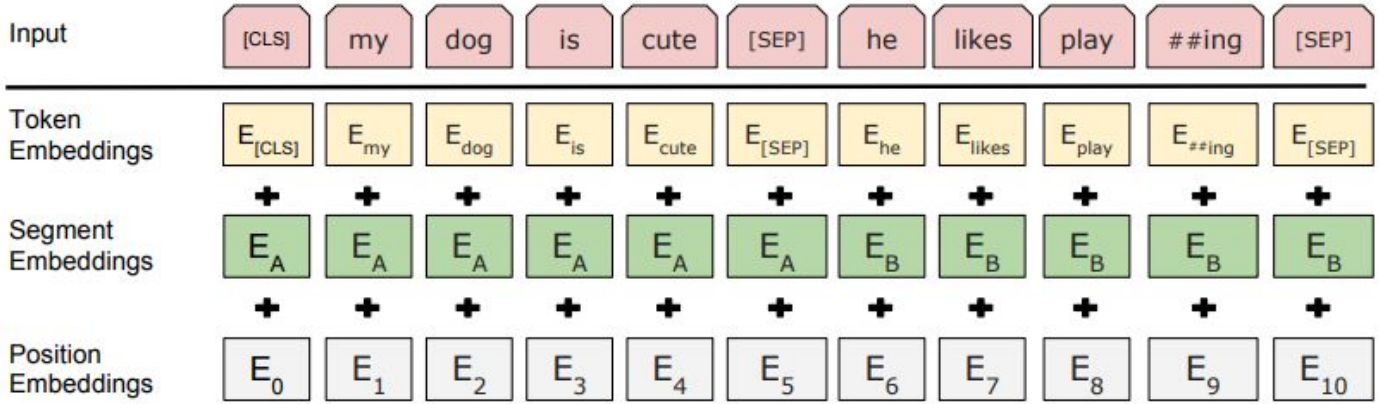
BERT 預處理



source:https://arxiv.org/pdf/1810.04805.pdf

BERT 預處理

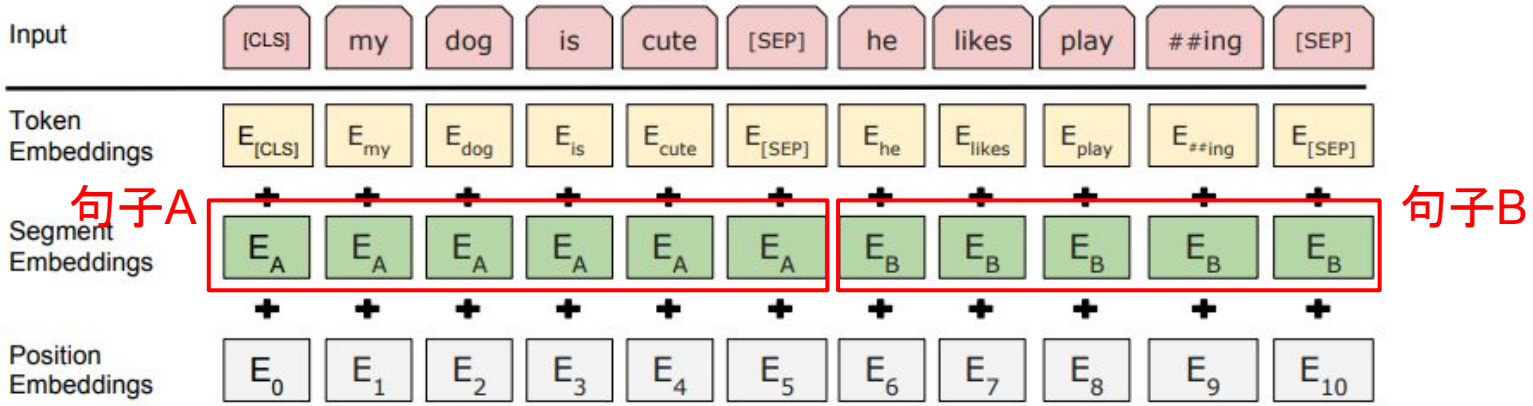
Token Embeddings：最淺層嵌入，代表詞語的淺層特徵，沒有上下文脈絡



source:https://arxiv.org/pdf/1810.04805.pdf

BERT 預處理

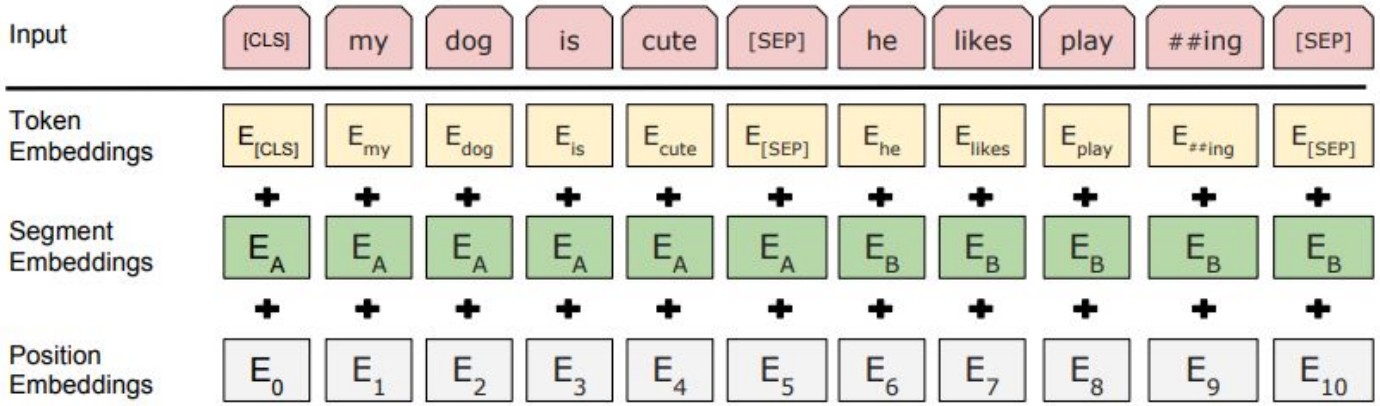
Segment Embeddings：BERT 可以將句子以成對的方式作為任務的輸入(例如問答)第一個句子的每個詞對應相同的Segment Embedding，第二句的每個詞對應第二種



source:https://arxiv.org/pdf/1810.04805.pdf

BERT 預處理

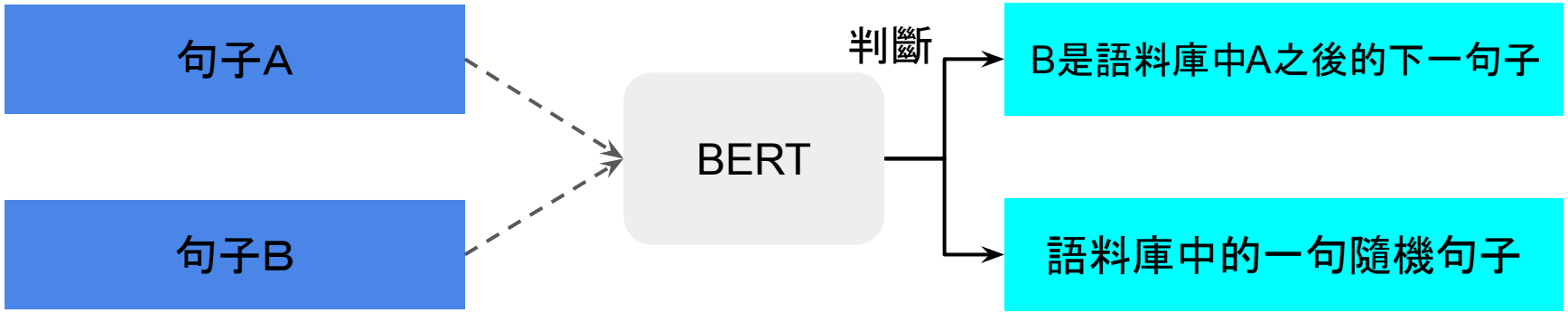
Position Embeddings：BERT 學習並使用位置嵌入來表示單詞在句子中的位置。添加這些功能是為了彌補 Transformer 的限制。與像 LSTM 這樣的 RNN 不同，BERT 同時對輸入進行**平行計算**，無法自然獲取順序或序列資訊，因此需要這種特殊手段來增強其性能。



source:https://arxiv.org/pdf/1810.04805.pdf

BERT 預訓練

- **下一句子預測 (Next Sentence Prediction, NSP)**
- 針對理解句子之間關係的任務，問答系統很適合詮釋此種任務的例子



BERT 預訓練

- **遮罩語言模型 (Masked Language Modeling, MLM)**
- 訓練一個語言模型來預測字詞時，從「序列本身」去預測缺失的單詞
- 因此，將某些字詞替換為[MASK]，在對模型進行訓練，使其預測這個缺失的詞

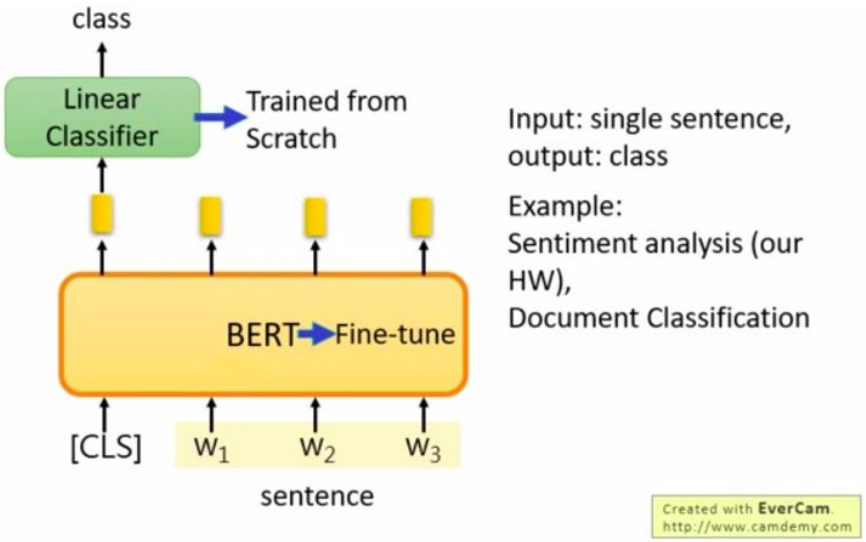
「今天的陽光溫暖而明媚，讓人感到無比愉悅。」

↓

「今天的[MASK]而明媚，讓人感到無比愉悅。」

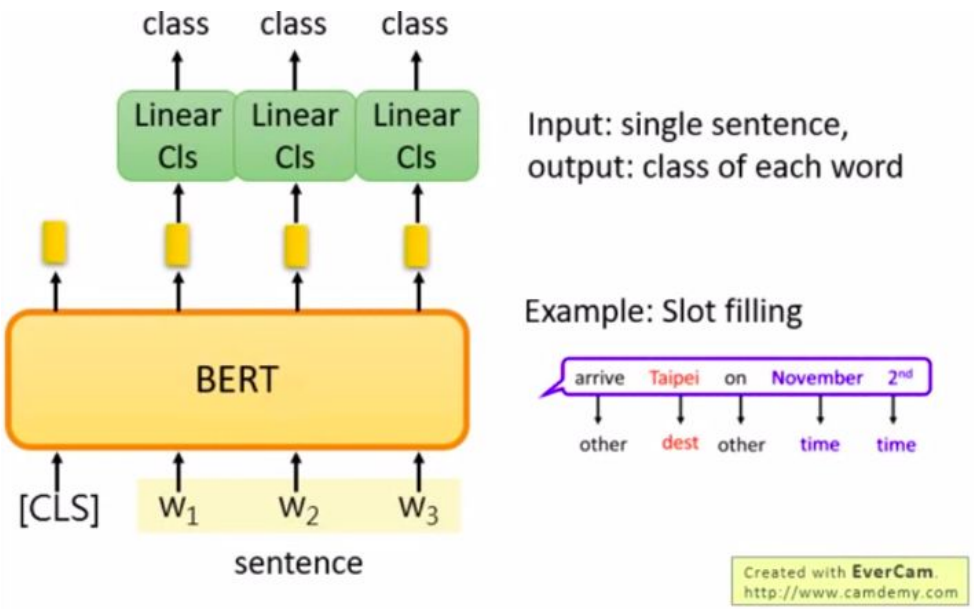
BERT 微調

- 句子分類判斷，如：情緒分析、文章分類
- 將想要分類的句子輸入BERT後，開頭加上代表分類符號[CLS]，再將該符號位置的輸出丟給線性分類器，由線性分類器去預測句子類別，只需微調BERT和訓練分類器



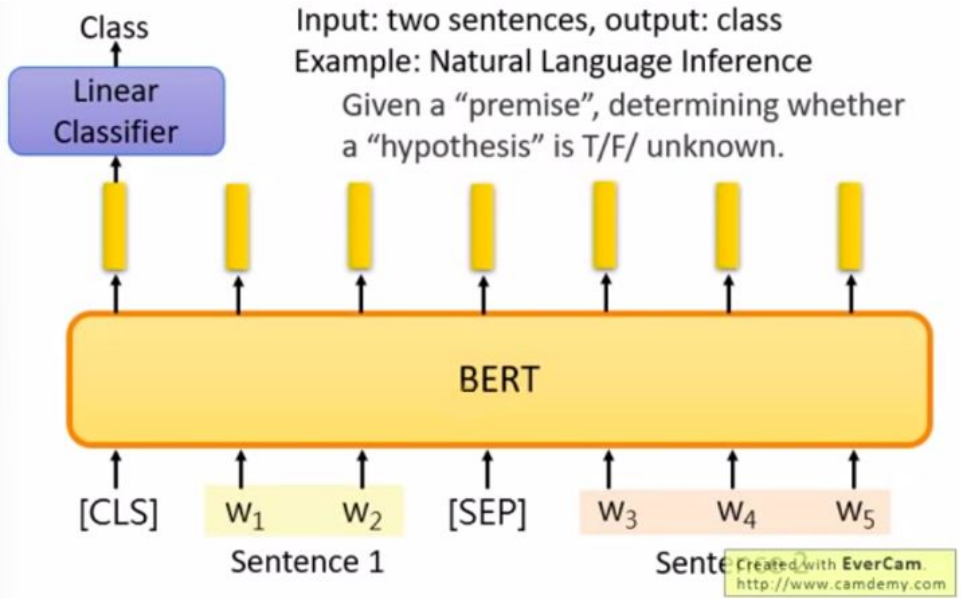
BERT 微調

- 將句子中每個詞分類:句子裡的每一個詞彙都要決定屬於哪個類別
- 輸入一個句子，再將每個詞彙的輸出都丟入線性分類器裡，讓其決定這個詞彙所屬類別



BERT 微調

- 判斷句子關係:讓機器學習推論一句話，給機器一個前提或假設，根據前提或假設推論對錯
- 輸入兩個句子，句子相接處放入[SEP]再將開頭處的輸出丟給分類器判斷



BERT 的問題

- BERT在預訓練時會出現[MASK]，而在微調中並不會出現，因此造成預訓練與微調不匹配
- 句子長度限制為512，超過長度的文本只能截斷
- BERT的訓練數據中只有15%的標記被預測，被預測的詞有80%被mask、10%替換成隨機詞、10%保持不變，導致模型需要更多訓練步驟來收斂

The quick brown fox jumps over the lazy dog

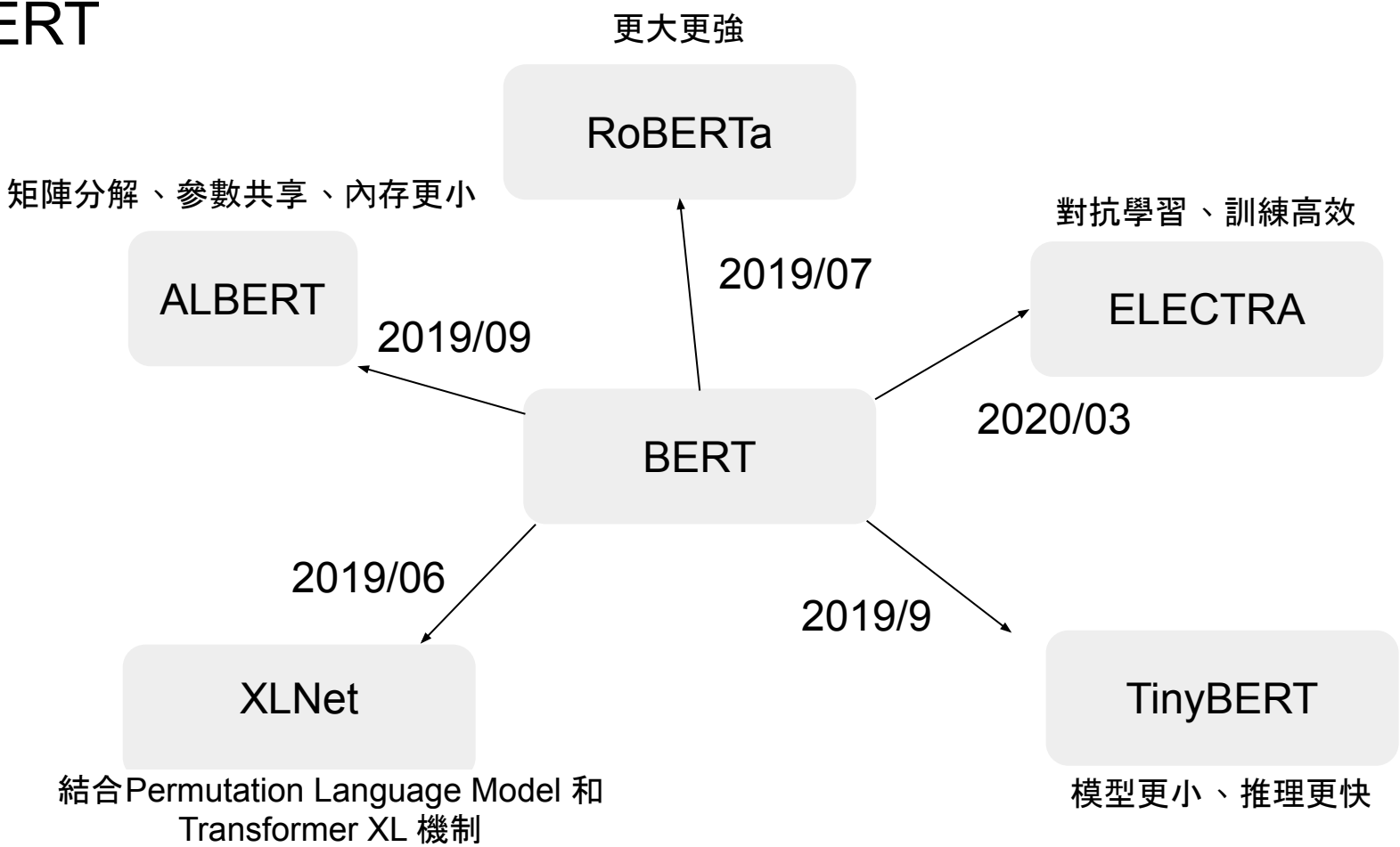
↓ 挑出序列15%固定的詞被選為是否 mask

The quick **brown** fox jumps over the lazy dog

↓ “brown” -> [mask] (80%機率)
“lazy” -> crazy (10%機率)

The quick [mask] fox jumps over the crazy dog

BERT


更大更強
RoBERTa
2019/07
矩陣分解、參數共享、內存更小
ALBERT
2019/09
對抗學習、訓練高效
ELECTRA
2020/03
BERT
2019/06
XLNet
結合Permutation Language Model 和
Transformer XL 機制
2019/9
TinyBERT
模型更小、推理更快

XLNet

- XLNet引入了排列語言模型(Permutation Language Model)，通過對輸入序列進行不同的排列組合，生成多種可能的排列順序，在訓練時能夠同時考慮詞的上下文訊息
  - 與BERT不同，BERT透過MLM捕捉上下文，而XLNet通過對輸入序列的排列組合進行訓練，避免了MLM的侷限性
  - 例如：輸入句子"A B C D"，XLNet會考慮"A C D B"、”B C A D”、"B D C A"等24種排列

Input: "The cat sat on the mat"

MLM：

"The cat [mask] on the mat"

PLM：

Permute to “sat cat mat The on the”
Predict "sat" | Context: {}
Predict "cat" | Context: {sat}
Predict "mat" | Context: {sat cat}
Predict "The" | Context: {sat cat mat}
Predict "on" | Context: {sat cat mat The}
Predict "the" | Context: {sat cat mat The on}

XLNet

- XLNet引入了排列語言模型(Permutation Language Model)，通過對輸入序列進行不同的排列組合，生成多種可能的排列順序，在訓練時能夠同時考慮詞的上下文訊息
  - 與BERT不同，BERT透過MLM捕捉上下文，而XLNet通過對輸入序列的排列組合進行訓練，避免了MLM的侷限性
  - 例如：輸入句子"A B C D"，XLNet會考慮"A C D B"、"B C A D"、"B D C A"等24種排列

Input: "The cat sat on the mat"

MLM：

"The

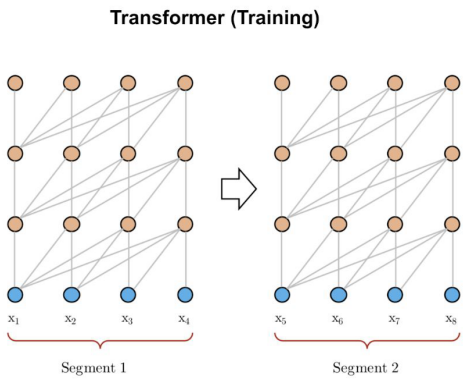
Note: 並不是真的改變字的順序，而是透過Attention Mask 來模擬排列組合的預測順序 (skip)

PLM：

Permute to "sat cat mat The on the"
Predict "sat" | Context: {}
Predict "cat" | Context: {sat}
Predict "mat" | Context: {sat cat}
Predict "The" | Context: {sat cat mat}
Predict "on" | Context: {sat cat mat The}
Predict "the" | Context: {sat cat mat The on}

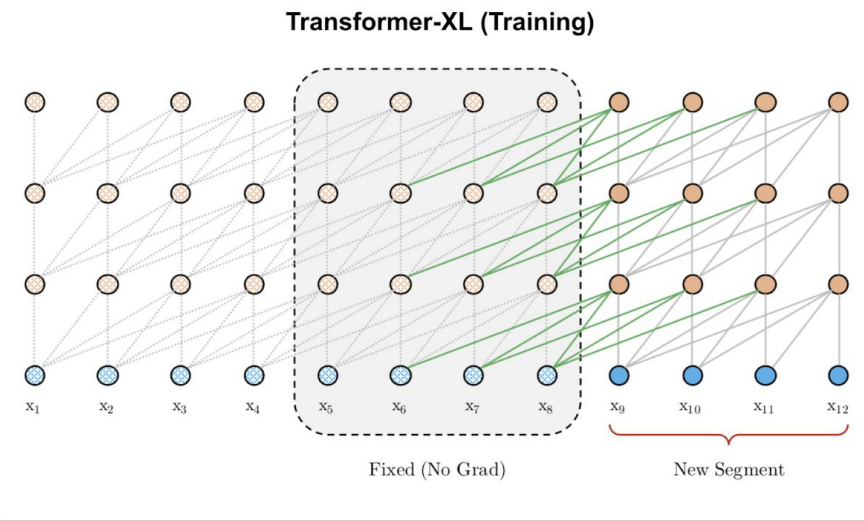
XLNet

- 利用Transformer-XL機制解決傳統Transformer模型處理長序列時的侷限性
  - 傳統Transformer是一次性處理一段固定長度的文本，例如 512個字，然後在處理下一段 512個字，這兩段之間沒有連續性 (Context Fragmentation)



XLNet

- 利用Transformer-XL機制解決傳統Transformer
  - 傳統Transformer是一次性處理一段固定長度的文
    ，這兩段之間沒有連續性 (Context Fragmentatic
- Transformer-XL的改進
  - **分段遞迴機制 (Segment-level Recurrence Mechanism)**：將文章分成多個片段，在處理第二段時，模型還會處理前一段 產生的隱藏狀態(記憶)，因此第二段的輸出將會考慮上一段的 內容



XLNet

- 利用Transformer-XL機制解決傳統Transformer模型處理長序列時的侷限性
  - 傳統Transformer是一次性處理一段固定長度的文本，例如 512個字，然後在處理下一段 512個字，這兩段之間沒有連續性 (Context Fragmentation)
- Transformer-XL的改進
  - **分段遞迴機制 (Segment-level Recurrence Mechanism)**：將文章分成多個片段，在處理第二段時，模型還會處理前一段 產生的隱藏狀態(記憶)，因此第二段的輸出將會考慮上一段的 內容
  - **相對位置編碼 (Relative Positional Encoding)**：每個詞的位置是相對於其他詞的位置，與傳統的絕對位置編碼不同

“我 喜歡 吃 蘋果”
"我"與"喜歡"的距離是+1
"我"與"蘋果"的距離是+3
⋮

句1：我 喜歡 吃 蘋果 → "喜歡" 是位置2，"蘋果" 是位置4
句2：我 討厭 吃 蘋果 → "討厭" 是位置2，"蘋果" 是位置4

絕對位置編碼
→ 模型只能記住「第 2個字 + 第4個字」的關係，what if 我 超級 無敵 喜歡 吃 蘋果？

相對位置編碼
→ 無論哪一句，「動詞」與「蘋果」的距離都是 +2
→ 模型能學到「動詞影響後面第 2個詞」，而非只記住在第幾位上出現
→ 可類推至「其實 我 超級 愛 吃 鳳梨」這樣的新句子

XLNet

- 利用Transformer-XL機制解決傳統Transfo
  - 傳統Transformer是一次性處理一段固定長
    , 這兩段之間沒有連續性 (Context Fragm
- Transformer-XL的改進
  - **分段遞迴機制 (Segment-level Recurrence Mechanism)**: 將文章分成多個片段, 在處理第二段時, 模型還會處理前一段 產生的隱藏狀態(記憶), 因此第二段的輸出將會考慮上一段的 內容
  - **相對位置編碼 (Relative Positional Encoding)**: 每個詞的位置是相對於其他詞的位置, 與傳統的絕對位置編碼不同

補充: 分段遞迴機制必須搭配相對位置編碼, 否則新舊 segment 的絕對位置會產生衝突

“我 喜歡 吃 蘋果”
"我"與"喜歡"的距離是+1
"我"與"蘋果"的距離是+3
⋮

句1: 我 喜歡 吃 蘋果 → "喜歡" 是位置2, "蘋果" 是位置4
句2: 我 討厭 吃 蘋果 → "討厭" 是位置2, "蘋果" 是位置4

絕對位置編碼
→ 模型只能記住「第 2個字 + 第4個字」的關係, what if 我 超級 無敵 喜歡 吃 蘋果？

相對位置編碼
→ 無論哪一句, 「動詞」與「蘋果」的距離都是 +2
→ 模型能學到「動詞影響後面第 2個詞」, 而非只記住在第幾位上出現
→ 可類推至「其實 我 超級 愛 吃 鳳梨」這樣的新句子

RoBERTa

在BERT的基礎上做了調整

- 增大batch-size、訓練數據更多（提升模型收斂穩定度與最終表現）
- static masking 換成 dynamic masking
  - **static masking**：BERT在處理訓練數據時，會先處理好哪些位置要被 mask，因此訓練時同一句話被mask的位置跟方式都是一樣的
  - **dynamic masking**：RoBERTa在訓練時才進行實時 mask，所以訓練時同一句話 mask的位置跟方式都是隨機的，存在多種可能
- 移除NSP任務(下一句子預測)，RoBERTa發現NSP對模型的效果沒有提升，只保留MLM

分詞的優化

- 在開始訓練之前，模型需要先建立Tokenizer / Vocabulary將文字轉成ID
- 將Character-level 分詞 (WordPiece)換成 Byte-level BPE(Byte Pair Encoding)
  - 在 BERT 的分詞過程中，單詞 "word" 會先被拆分為 "w"、"o"、"r"、"d" 這四個字元。接著，分詞器會根據語料中最常見的相鄰字元對（例如 "wo"）進行合併，變成 "wo"、"r"、"d"。這個合併過程會重複進行，直到達到預設的合併次數或詞彙表大小。最終， "word" 可能會被切分為 "wo" 和 "rd" 這樣的子詞。即使如此，所有最基本的字元（如 "w"、"o"、"r"、"d"）也會保留在詞彙表中，以保證能處理任何新詞。
  - 在RoBERTa中會以UTF-8編碼儲存，例如漢字 "哭"的UTF-8編碼是三個字節 (0xE5,0X93,0xAD)，再進行Byte-level BPE時，每個字節被視為 1個token:"E5","93",”AD”，在訓練的BPE詞彙表查詢這三個序列是否常出現在數據集，並決定是否合併

Character-level BPE VS Byte-level BPE

- Character-level BPE 的缺點:
  - 遇到未見過的字元(特別是 Unicode 字元)時, 無法處理, 且詞彙表需要包含所有可能的字元, 導致詞彙表膨脹。
- Byte-level BPE 的優點:
  - 因為是以 byte 為單位, 所以可以處理 Unicode 中任意語言, 包含 emoji、罕見字等
  - 詞彙表只需包含 256 個基本 bytes, 節省空間, 且不會出現無法編碼的情況